

Faro — Technical manual

How it works inside: architecture, algorithms, formulas and design decisions.

Written for whoever will maintain this code. Every claim points at the file and line where it lives, and what is an assumption is called one.

1. What this is, and how it is split

Faro turns a distributor's sales history into the purchase decision of the day: what to order, how much, from whom, and when it has to be placed so it arrives before the stock runs out. This document describes how it does that inside. It is written against the code, with file and line, and where a number is an assumption it says so.

The whole chain

The entire product is a chain of five links. Each takes something measured and returns something more useful, and at every joint there is an assumption worth knowing.

HOW IT WORKS

- Sales history (CSV/Excel/SQL) → a canonical table with sku, date and demand.
- Per SKU: series classification, model selection, walk-forward training, champion selection.
- Forecast with an interval → expected daily demand across the lead time.
- Against today's stock: coverage days → stock signal → recommended quantity.
- Purchase order, sent to the supplier, received back, and the real lead time learned.

WHAT IT DOES NOT MEAN

- The last link is what closes the loop: without recording the order and its reception, the supplier's real lead time is never learned and the system keeps planning on what was configured.

The three layers

[ForecastingCore/](#) · [backend/](#) · [Frontend/](#)

A strict separation with a reason: the engine can be sold, tested and replaced without touching the product.

ForecastingCore/	ALL forecasting intelligence. It does not know the database and does not know tenants. It takes a config dict and returns dicts and DataFrames.
backend/	Multi-tenant FastAPI API. Pure orchestration: no ML, and it may not import pandas or numpy outside three modules — a test enforces it.
Frontend/	Next.js 14. No business logic: it asks, shows and translates.

WHAT IT DOES NOT MEAN

- The engine NEVER writes to a database. What persists is backend/workers/runner.py, which is also the only bridge between the two layers.

What decides what

Four decisions govern everything else. When the product surprises you, it is almost always one of these.

The series classification	Decides which models get trained. A short series (< 20 observations) is dropped entirely.
----------------------------------	---

The per-SKU champion	Decides which forecast is used downstream. The rule lives in a constant shared verbatim between the engine and the backend, because when they drifted the two layers disagreed on 8 of 13 products.
The lead time	Decides all three signal thresholds AND the replenishment-period demand. It is the most sensitive number in the system, and its default is an assumption of 15 days.
The service level	Decides the cushion. Default 0.95, resolvable per SKU, supplier, category or globally.

How to read the rest of this

Every topic carries the same four things: what it is, how it works, the formula where there is one, and — the section that matters most — what it does NOT mean.

WHAT IT DOES NOT MEAN

- The warnings are not decorative self-criticism. They are the things an engineer finds in the first week; finding them unannounced is what makes the rest of a document read as marketing.
- Where a constant is hard-coded and not configurable, it says so. Where a parameter exists and nothing reads it, it says that too.

2. Ingestion: from the customer's file to a modellable series

Faro does not ask for a format. It reads the export the customer already has — CSV or Excel, with varying separators and encodings, Spanish headers and dd/mm/yyyy dates — and turns it into a canonical table. This is the part with the most field work in it and the cheapest to underestimate: almost every real file arrives broken in some way.

Reading the file

[forecasting_core/data/loader.py:109](#)

One entry point for CSV, XLSX, XLS, Parquet, JSON and SQL. What looks trivial is where real files get lost.

HOW IT WORKS

- The separator is sniffed from the header among comma, semicolon, tab and pipe (loader.py:17).
- Encodings are tried in order: utf-8-sig, cp1252, latin-1 (loader.py:37). Excel's BOM and Windows cp1252 are the norm, not the exception.
- A date column that is unambiguously dd/mm/yyyy is parsed day-first (loader.py:62). Ambiguous ones are not guessed.

WHAT IT DOES NOT MEAN

— A file containing a NUL byte is refused in backend/dataframes/io.py, not here: pandas truncates the cell at the NUL without a word, and a silent truncation is worse than a refusal.

Canonical schema

[forecasting_core/data/canonical.py:49](#)

Fourteen canonical fields the user's columns are mapped onto. Three are required: sku, date, demand. The rest carry a default that is broadcast across the column when the file lacks it.

sku, date, demand	Required. Without them there is no series.
store	Defaults to "Tienda única". The series key is sku store, separator U+2502 (canonical.py:66).
region	Defaults to "Sin región".
inventory	Defaults to 0. When the file really carries it, censored-demand recovery switches on.
lead_time	Defaults to 15 days (DEFAULT_LEAD_TIME_DAYS, canonical.py:23). It is an ASSUMPTION, and the product says so on screen.
price, cost, regular_price, promo_price	Default to None.
promo, promo_type, discount	Default to False, "Sin promoción", 0.0.

WHAT IT DOES NOT MEAN

— The 15-day default lead time is the most expensive assumption in the system: an importer with 45 days of transit reorders a month late and nothing visibly fails.

Gap filling and outliers

[forecasting_core/validation/auto_correct.py:100](#)

Two automatic corrections before modelling, and one distinction that matters: filling a null TARGET is not the same as filling a missing DATE.

HOW IT WORKS

- Null target: per group, ffill → bfill → fillna(0) (auto_correct.py:126). Always on.
- Outliers: per-SKU clip to [Q1 − 3·IQR, Q3 + 3·IQR] (auto_correct.py:139). The pipeline passes clip_outliers=True unconditionally (pipeline.py:279).
- Missing dates: the engine does NOT fill them. The strategies (zero, forward fill, interpolate, leave) live in the gate catalogue (data/gate.py:107) and are applied by backend/workers/runner.py; the default is "leave".

WHAT IT DOES NOT MEAN

- auto_correct.py:104 declares a fill_missing_dates parameter and NEVER uses it. It is dead code: anybody reading it and assuming dates get filled is wrong.
- DataQualityChecker.clip_outliers (quality.py:223) exists and the pipeline never calls it. The same 3.0 factor IS used to COUNT outliers for the quality score.

Censored demand

[forecasting_core/data/censoring.py](#)

A day with no sales because there was nothing on the shelf is not a day of zero demand. If the customer mapped a real inventory column, those observations are lifted; if they did not, this step is a no-op.

LEVEL_WINDOW	28 — window used to estimate the uncensored demand level.
MIN_UNCENSORED	14 — minimum uncensored observations before estimating.
MAX_LIFT_FACTOR	3.0 — ceiling on the adjustment. It never multiplies a sale by more than three.
demand_censored	Flag column left on the table.

WHAT IT DOES NOT MEAN

- Observations are only ever RAISED, never lowered. It is an upward bias correction and you have to know that when reading a forecast.

Series classification

[forecasting_core/data/quality.py:80](#)

Every SKU gets labels — it is MULTI-LABEL, a series can be seasonal and volatile at once. Which models get trained for it depends on this.

HOW IT WORKS

- If $n < \text{min_history}$ (20) $\rightarrow$ {short}, returning immediately without evaluating anything else (quality.py:101).
- If the zero ratio $\geq 0.4 \rightarrow$ intermittent.
- If $\text{CV} = \text{std}/(|\text{mean}|+1\text{e-}8) \geq 1.5 \rightarrow$ volatile.
- If long enough, STL with period=7: $\text{strength} = \max(0, 1 - \text{var}(\text{resid})/(\text{var}(\text{seasonal}+\text{resid})+1\text{e-}8))$; $> 0.3 \rightarrow$ seasonal. Any exception is swallowed and the seasonal flag is simply not set.
- No flag set $\rightarrow$ stable.

Seasonal strength

$\text{strength} = \max(0, 1 - \text{var}(\text{resid}) / (\text{var}(\text{seasonal} + \text{resid}) + 1\text{e-}8))$

Robust STL, period 7 by default (training.seasonal_period).

Quality score

$100 - 30 \cdot (n < 20) - \min(\text{missing} \cdot 2, 20) - \min(\text{outliers} \cdot 3, 15) - \min(\text{zero_ratio} \cdot 20, 20)$

Floored at 0, published divided by 100 (quality.py:266).

WHAT IT DOES NOT MEAN

- filter_valid_skus drops every SKU with fewer than 20 observations. A new catalogue can lose half its products, which is correct behaviour but has to be explained to the customer.
- A legacy classify_series (quality.py:53) exists with its own thresholds and the pipeline does NOT use it.

Granularity and aggregation

[forecasting_core/data/resampler.py:19](#)

The default strategy is "native": it touches nothing. With "aggregate" the series is resampled to a target frequency, summing the target and aggregating other columns per a dict (inventory is carried with min).

WHAT IT DOES NOT MEAN

- A limitation the code states itself (pipeline.py:181): resampling uses only the PRIMARY group key, so a two-key configuration sums across the secondary one.
- There are two different frequency-detection ladders with different bounds — profiler.py:911 and profiler.py:935 — plus infer_freq_days in quality.py:27. They are not unified.

3. Forecasting: from a series to a number with an interval

The engine lives entirely in ``ForecastingCore/`` and never touches the database: it takes a configuration, returns dicts and DataFrames, and the backend persists them. Training is per SKU with walk-forward validation, and the champion is chosen by a rule the backend shares verbatim — when the two drifted, the layers disagreed on 8 of 13 products.

Entry point and pipeline order

[forecasting_core/engine.py:68](#) → [pipelines/pipeline.py:201](#)

``ForecastEngine`` exposes a fluent API; ``Pipeline.run()`` does the work in 16 stages. The only bridge from the backend is ``backend/workers/runner.py:1312``.

HOW IT WORKS

- File read → validations (WARNING mode, they never abort) → auto-correct → censored demand.
- Per-SKU quality, series classification, and dropping series that are too short.
- Model routing → feature engineering → baselines.
- Per-SKU training with walk-forward validation, quantile models, the global model, the statistical models.
- Weighted ensemble → metrics table → future forecasts → inventory recommendations → run registry.

WHAT IT DOES NOT MEAN

- Hierarchical reconciliation (`bottom_up` / `top_down` / `mint`) exists but is OFF by default: `hierarchy.levels` arrives empty ([config.py:164](#)).
- The engine never writes to a database. `ModelRegistry.log_run` writes a JSON file on disk (`registry.json` by default).

Model routing

[forecasting_core/training/router.py:30](#)

The series classification decides which models are worth training for it. The rule that governs everything: routing may only NARROW the user's selection, never widen it.

The rule

`run = assigned & declared # intersection, never union`

``declared`` is the user's selection, verbatim. If the intersection comes out empty it falls back to the full ``declared`` rather than leaving the SKU with no model ([router.py:80](#)).

short	naive, seasonal_naive
intermittent	croston, ets
seasonal	prophet, ets, lightgbm, sarimax, lstm

stable	lightgbm, xgboost, arima, sarimax, lstm
volatile	lightgbm, xgboost, prophet, arima, ets, sarimax

WHAT IT DOES NOT MEAN

- Labels are unioned before intersecting: a {seasonal, volatile} series gets the union of both rows.
- REAL GAP: `naive` and `seasonal\_naive` appear in the table for short series but do NOT exist as trainable models in ModelFactory — only as baselines. A short series therefore always hits the empty-intersection fallback.
- `global\_lgbm` is in UNIVERSAL_MODELS and bypasses the narrowing, but is still gated on the user having declared it.

Implemented models

[forecasting_core/models/factory.py:91](#)

Nine trainable models plus three baselines. Parallelism lives in the SKU loop, which is why every model runs with `n_jobs=1`.

lightgbm / xgboost	<code>n_estimators</code> 300, <code>learning_rate</code> 0.05 by default. The only two with an Optuna search space.
global_lgbm	One LightGBM over the whole catalogue, direct multi-horizon (the horizon is a feature). 400 trees, 63 leaves. Stacked matrix capped at 2,000,000 rows.
arima	<code>statsmodels</code> , order (5,1,2). Refits on the full series for the future forecast; <code>conf_int(alpha=0.2)</code> gives p10/p90.
sarimax	Order (1,1,1), seasonal (1,1,1,period). Only runs when the user selected it explicitly.
prophet	Yearly and weekly seasonality on, daily off.
ets	Additive trend; additive seasonality only with 2 full periods and an all-positive series.
croston	Classic Croston, alpha 0.1. Not SBA, not TSB.
lstm	TensorFlow is OPTIONAL: without it the model returns empty with a logged warning. Window 14, 50 epochs, early stopping.
baselines	<code>naive</code> , <code>seasonal_naive</code> , <code>historical_avg</code> . Scored for comparison and EXCLUDED from being champion.

WHAT IT DOES NOT MEAN

- The session schema that exposes the configuration (`config.py:348`) lists 8 models and OMITS `global_lgbm`, while ModelFactory has it. The backend's own default selection does include it.
- Optuna is off by default and only has a search space for `lightgbm` and `xgboost`; any other name gets its parameters back untouched.

Features and the leakage guard

[forecasting_core/features/engineer.py:47](#)

Calendar, optional Fourier, lags, diffs, rolling windows and EWM. Everything backward-looking is built on `shift(1)` so a row cannot reconstruct its own target.

HOW IT WORKS

- Calendar: 19 columns, the same builder for training and inference. Holidays per country from the ``holidays`` library, default country CO; without the library it degrades to Easter and Christmas, never to an empty set.
- Lags [1, 7, 14]; diffs [1, 7] computed on `shift(1)`; rolling [7, 14, 28] with mean, std, min, max and CV.
- The warm-up `dropna` runs ONLY over generated columns and the target — a blanket `dropna` once emptied whole datasets.

Leakage guard (trainer.py:266)

```
leak_cols = [c for c in X if is_numeric(X[c]) and X[c].astype(float).equals(y)]
```

Any column identical to the target is dropped with a warning naming the SKU. It targets the canonical ``demand`` alias, a literal copy of the user's mapped column.

WHAT IT DOES NOT MEAN

- ``_drop_unservable_features`` removes columns present in history but not reproducible for a future date (inventory, price, cost).
- PARTIALLY INERT: `validation/leakage.py` is called without ``feature_columns``, so only the unsorted-dates check fires. The correlation-based leakage checks (> 0.999) are unreachable from the pipeline.

Validation and champion selection

[forecasting_core/pipelines/pipeline.py:794](#)

Walk-forward validation with an expanding window and a GAP equal to the horizon, so the training window never touches data the production forecast could not have had.

HOW IT WORKS

- Per-SKU ML: 3 splits by default, minimum training window 40% of the series, `gap = horizon`. With no viable splits it falls back to a single chronological split.
- Two scores per model, deliberately: fold metrics are 1-step, and ``horizon_metrics`` runs the REAL inference path forward from the final cutoff.
- Global model: rolling origin over the date axis, refitting at each cutoff.
- Statistical models: a single chronological split; their cost is already an h-step number.

Primary metric

$$\text{WAPE} = \sum |y - \hat{y}| / (\sum |y| + 1e-8)$$

Also: MAE, RMSE, bias, MAPE, sMAPE, pinball.

Asymmetric cost

$$\text{cost} = \text{mean}(\text{over} + 3.0 \cdot \text{under})$$

`DEFAULT_STOCKOUT_MULTIPLIER = 3.0`. Running out costs three times what sitting on stock does.

Champion rule

```
metric = first of (cost_horizon, cost, wape, mae) present; champion = argmin(metric)
```

Baseline rows are excluded: a naive forecast can never be crowned. No explicit tie-break.

WHAT IT DOES NOT MEAN

- If the best baseline beats every model, the SKU is recorded in ``outperformed_by_baseline`` and still gets its best real model.
- The ensemble emits a forecast but has NO metrics row, so it can never be champion.
- MASE and CRPS are not implemented anywhere.
- `CHAMPION_METRIC_ORDER` is shared verbatim with `backend/inventory/service.py::best_model_by_sku`. Letting them drift made the two layers disagree on 8 of 13 SKUs.

Prediction intervals

[forecasting_core/inference/predictor.py:29](#)

Three different mechanisms, and which one you get depends on the model. This matters more than it looks: the inventory layer extracts its sigma from these intervals.

Normal approximation	<code>bound = max(0, value + z_q * std(residuals))</code> . Out-of-fold residual bank when available. This is the path for LightGBM/XGBoost without quantile models, and for ETS, Croston and SARIMAX.
Quantile regressors	LightGBM/XGBoost trained at <code>q = 0.1/0.5/0.9</code> and predicted directly.
Per-horizon conformal	<code>global_lgbm</code> only. Empirical quantiles of the backtest residuals, computed SEPARATELY per horizon and pooled in scaled units. Minimum 30 residuals per horizon; below that the horizon borrows the pooled bank.
Model-supplied bands	ARIMA (80% CI), Prophet (<code>yhat_lower/upper</code>), LSTM (Monte-Carlo Dropout, 30 passes).

WHAT IT DOES NOT MEAN

- With no calibration data at all, the conformal path returns a DEGENERATE band (`lower = upper = value`) rather than fabricating a width.
- ``enforce_monotonic`` repairs crossed quantiles with a running max; ``enforce_horizon_monotonic`` makes cumulative bands non-decreasing in `h`.
- ``lead_time_demand_quantile`` gives the quantile of CUMULATIVE demand over the lead time: that is what the reorder point consumes.

4. Inventory: from a forecast to a purchase order

This is where the product lives. The forecast is an input; what the customer buys is "order 78 of these today". This whole chapter is four chained numbers — coverage, signal, reorder point, quantity — and the most important part is where each one stops being a measurement and becomes an assumption.

Coverage days

[backend/inventory/service.py:1492](#)

How long today's stock lasts at the forecast sales rate. Every other number hangs off this one.

Coverage

```
coverage_days = current_stock / avg_daily if avg_daily > 0, else 9999.0
```

`avg\_daily` is the mean of the first `steps` forecast points of the SKU's BEST model, with steps = `ceil(lead_time_days / days_per_period)`.

WHAT IT DOES NOT MEAN

- IT IS NOT IN DAYS, despite the name. It is in periods of the active planning grain: on a weekly session, "3" means 3 weeks. The authoritative field is `coverage\_unit`, shipped alongside.
- The numerator is on-hand stock only. Orders in transit do NOT enter here (they do enter the recommended quantity), so a SKU with a truck inbound can show PEDIR_YA with quantity 0.
- Zero demand yields the 9999 sentinel, which classifies as SOBRESTOCK and is nulled before leaving: the customer sees `coverage\_days: null, signal: SOBRESTOCK`.
- If the best model stored no points, it silently falls back to averaging EVERY trained model.

The stock signal

[backend/inventory/service.py:895](#)

The whole rule, verbatim, is four lines.

Signal

```
coverage < lead_time*0.5 → PEDIR_YA coverage < lead_time*1.2 → PEDIR_PRONTO coverage < lead_time*3 → OK otherwise → SOBRESTOCK
```

The 0.5 / 1.2 / 3 multipliers are literals in that function.

WHAT IT DOES NOT MEAN

- There is NO per-tenant and NO per-SKU threshold configuration. The only thing that varies per product is the lead time.
- SIN_DATOS does not come out of this function: it is the branch taken when the forecast is missing or the stock row is. In the per-warehouse view it also fires when that warehouse's demand share is 0.
- Import forces lead_time_days >= 1: a stray 0 collapses all three thresholds and paints everything SOBRESTOCK.
- `min\_stock` is stored, imported and returned, but is read by NEITHER the signal NOR the recommendation. It is dead weight in this layer.

Recommended quantity

[backend/inventory/service.py:949](#)

Lead-time demand plus safety stock, minus what you have and what is already coming. With an MOQ floor and a gate on the signal.

Quantity

```
raw = max(0, avg_daily.lead_time + safety_stock - current_stock - incoming) if moq > 0
and raw > 0: raw = max(ceil(raw), moq)
```

It is then forced to 0 unless the signal is PEDIR_YA or PEDIR_PRONTO.

Reorder point

```
reorder_point = avg_daily.lead_time + safety_stock
```

It is published, but it is NOT the trigger: the trigger is the coverage ratio above.

Safety stock — measured (preferred)

cumulative offsets from the rolling-origin backtest, horizon `ceil(lead_time)`, quantile nearest the service level

Used only when the model that produced the risk is the SKU's champion; otherwise discarded.

Safety stock — classical fallback

```
safety = z · σ · sqrt(lead_time)
```

σ is extracted from the interval: $\sigma = (q90 - \text{value}) / 1.2816$.

WHAT IT DOES NOT MEAN

- The code itself says $z \cdot \sigma \cdot \sqrt{L}$ is wrong: it assumes per-bucket errors are normal and independent, and "neither holds: demand is non-negative and skewed, and a forecast that runs high today runs high tomorrow, so the errors compound faster than $\sqrt{L}$ ". The measured branch exists precisely to avoid it.
- MOQ is a MINIMUM, not a multiple: a need of 520 with MOQ 500 orders 520, not 1000. It used to order 1000.
- There is NO review period. The model is continuous-review, with the trigger replaced by the coverage ratio.
- The reorder point and the signal can contradict each other: when safety stock exceeds 20% of lead-time demand, a SKU can sit BELOW its own published reorder point and still be painted OK with quantity 0.

Learned lead time

[backend/inventory/service.py:1047](#)

Every completed reception leaves an observation of the supplier's real lead time. At three observations, the learned average replaces what was configured.

Learning

```
AVG(lead_time_days) GROUP BY LOWER(supplier) HAVING COUNT(*) >= 3
```

`MIN_LEAD_TIME_OBSERVATIONS = 3`. Below it the supplier is absent from the map entirely.

WHAT IT DOES NOT MEAN

- The learned value WINS unconditionally over everything configured, and applies to EVERY SKU of that supplier — including ones configured by hand, one at a time.

- It is a plain mean over every observation ever recorded: no recency weighting, no outlier rejection, no cap.
- Only a COMPLETE reception writes an observation. A partial one contributes nothing. And on a multi-supplier order, EVERY supplier is dated by the completion of the whole order.

Multi-warehouse and transfers

[backend/inventory/service.py:1682](#)

Demand is split across warehouses in one of two ways, and when one warehouse needs what another has spare, a transfer is proposed instead of a purchase.

HOW IT WORKS

- "store" mode: the session was trained with a store column, so each warehouse has its own real forecast. Share 1.0.
- "share" mode: the SKU-global forecast is multiplied by each warehouse's configured share. If none has one, 100% of demand goes to the default warehouse.
- A donor must keep at least 30 days of its own coverage and be able to cover at least 80% of the need.
- A lane is accepted if it is strictly faster than purchasing AND strictly cheaper; with no known unit cost it is accepted on time alone, and says so.

WHAT IT DOES NOT MEAN

- The aggregate status ("All") is NOT a rollup of the per-warehouse signals: it is an independent computation on summed stock against the un-split forecast. The two views can legitimately disagree on a SKU.
- An unconfigured lane is worth 1 day, cost 0 and fixed 0 — the module calls it "deliberately optimistic" itself. It beats almost any purchase. The `lane_is_default` flag rides along so the screen can say so.

The optimizer

[ForecastingCore/forecasting_core/business/optimizer.py:46](#)

A MILP solved with HiGHS (`scipy.optimize.milp`) that spreads purchases and transfers across the horizon, minimising total cost.

Objective

$\min \Sigma \text{holding} \cdot \text{inv} + \text{stockout} \cdot \text{short} + \text{order_cost} \cdot \text{order} + \text{transfer_cost} \cdot \text{transfer} + \text{fixed} \cdot \text{ship}$

Variables per bucket: order, transfer, inv, short (integer ≥ 0) and ship (binary, only on lanes with a fixed cost).

Costs

$\text{holding} = \text{unit_cost} \cdot 0.20 / 365 \cdot \text{days_per_bucket}$; $\text{stockout} = \text{order_cost} \cdot 3.0$

0.20 is the default annual holding rate; 3.0 the stockout multiplier.

WHAT IT DOES NOT MEAN

- A 10-second solver limit, and EVERY solve runs on one dedicated thread: entering HiGHS from different OS threads deadlocks, and a mutex does not fix it.

- On exception, timeout, or more than 5000 variables, it degrades to a naive fallback that IGNORES transfers entirely, reported as status "fallback".
- One concurrent solve only: a second caller gets a 503 and retries. At two, two buyers received two different plans for the same catalogue.
- A unit cost of 0 or negative is treated as missing and 1.0 is used, flagging the line as ``assumed_unit_cost``. The ``total_cost`` can be a number about nothing.
- MOQ is applied AFTER the solve, adding the shortfall to the largest line. The MILP has no minimum-order variable.

What the business figures do not say

[backend/inventory/roi_service.py:516](#)

The impact screen is deliberately poorer than it could be, and that is a decision rather than a gap.

WHAT IT DOES NOT MEAN

- ``capital_freed`` is the difference between two monthly overstock snapshots. Overstock also falls on sales, shrinkage, SKU deletion and retraining: the figure is NOT attributable to Faro, and the code says so.
- No "Faro saved you \$X" headline and no "stockouts avoided" count are computed, because both need assumptions not grounded in the tenant's own data.
- Dead stock is a heuristic: a SKU is "dead" when observed depletion is under 20% of forecast-expected depletion, with at least 2 snapshots and no restocking in the window.
- ABC uses daily demand $\times$ unit cost as a revenue proxy, falling back to a unit cost of 1.0 when none is on file. The XYZ cutoffs are CV 0.5 and 1.0, hard-coded.

5. The platform: multi-tenancy, jobs, configuration and API

Three layers with a strict separation, a job queue that IS a table, and tenant isolation that is query discipline rather than a database guarantee. This chapter says where the locks are, and also where they are NOT.

Layers and the one boundary a machine checks

[backend/tests/test_no_pandas_in_backend.py:33](#)

ML engine, orchestration API and UI. Of the three rules, only one is mechanically enforced: pandas and numpy may not be imported in `backend/` outside three modules.

Allowed	everything under backend/dataframes/, backend/utils/temporal_agg.py and backend/workers/runner.py
What crosses	A path or bytes go in; plain Python comes back — list[dict], dict, list[str], with NaN turned into None
The four exceptions	read_dataframe, dataframe_from_records, filter_dataframe_by_date and last_row_per_group: the only points where a DataFrame reaches the engine, each labelled as such

WHAT IT DOES NOT MEAN

- There is NO equivalent test forbidding business logic in the frontend or ML in the backend. That half is convention.

Multi-tenancy

[backend/auth/guards.py:277](#)

The tenant travels in the JWT and is enforced query by query. Every service function takes `tenant\_id` as its first argument and every statement carries `AND tenant\_id = %s`.

WHAT IT DOES NOT MEAN

- There is NO row-level security in the database. Isolation is discipline, not a guarantee: one forgotten `AND tenant\_id` breaks it.
- The tenant-context middleware is observability ONLY — its own docstring says so. The authority is `CurrentUser.tenant_id`, which is never read from a request body.
- The safety net is tests that read reality: one walks FastAPI's own route table and fails any mutation without a guard; another reads the Postgres catalog and demands that every table with a `tenant_id` has an ON DELETE CASCADE FK (born of 1.3M orphan rows across 24,794 dead tenants).
- An id belonging to another tenant returns 404, not 403: it does not confirm the row exists.

Authentication

[backend/auth/jwt_handler.py](#)

Its own JWT, not Supabase. Two distinct credentials, one human and one machine.

Access token	15 minutes, HS256, claims sub/tenant_id/role/email_verified/jti.
---------------------	--

Refresh token	7 days. NOT a JWT: 64 random bytes stored only as SHA-256. At most 5 live per user.
Revocation	Two mechanisms: a jti blocklist for logout, and a time cut (sessions_invalid_before) for password changes, which cannot know a jti.
API keys	sk_live_ + 32 bytes. Unsalted SHA-256 (32 bytes of entropy needs no salt). They can never be admin: viewer or analyst only.
Key limits	120 per minute plus the plan's daily ceiling, decided under an advisory lock — count-then-insert let 16 of 20 concurrent calls past a ceiling of 5.

WHAT IT DOES NOT MEAN

- CHECK THIS: `ACCESS\_TOKEN\_EXPIRE\_MINUTES` exists, is documented and is registered... and nothing reads it. The 15 minutes are a constant in the code. Changing that variable does nothing.
- Key rate limiting FAILS OPEN on a database error: serving is preferred to blocking. Deliberate and documented.
- A key acts with ITS own role, not its creator's, and each of its writes leaves an audit row with actor `api\_key:<id>`.

Jobs and the worker

[backend/training/queue.py:50](#)

The queue is the `jobs` table. `enqueue()` is an explicit no-op: inserting the row IS enqueueing.

The claim

```
UPDATE jobs SET status='RUNNING', worker_id=%s WHERE id = (SELECT id FROM jobs WHERE
status='QUEUED' ORDER BY created_at FOR UPDATE SKIP LOCKED LIMIT 1)
```

SKIP LOCKED is what makes two replicas take DIFFERENT jobs. It replaced a read-then-write that trained the same session twice.

job-worker	Polls every 2 s while there is room. Concurrent jobs per process: 2.
job-scheduler	Every 60 s: due scheduled jobs, with cron evaluated in the tenant's timezone.
integration-sync	06:00 UTC. Before the alerts, deliberately.
inventory-alerts	08:00 UTC: stockout digest, supplier lead-time deviations and stale-data reminders.
overstock-snapshot	Day 1, 00:05 UTC: overstock snapshot, then the monthly recap emails. The order is load-bearing.

WHAT IT DOES NOT MEAN

- SCHEDULER_ENABLED must be on in EXACTLY ONE instance or every daily alert goes out twice.
- Orphan recovery runs at WORKER startup, not the API's: an API redeploy must not kill another process's live trainings.
- CHECK THIS: the per-process job cap is 2 and the per-plan cap is 8. In a single-process deployment the 2 binds first.

Session state and the six config blobs

[backend/sessions/state_machine.py:26](#)

DRAFT → ... → MODELS_CONFIGURED → QUEUED → RUNNING → COMPLETED/FAILED. Six JSONB blobs in `session\_configs` that the runner assembles into one dict for the engine.

columns_cfg	Mapping of the user's columns onto the canonical schema.
features_cfg	Lags, windows, calendar, holiday country.
models_cfg	The models the user selected — the selection routing may only narrow.
validation_cfg	Split, folds, horizon.
forecast_cfg	Horizon and quantiles.
business_cfg	Service level 0.95, lead time, 20% holding, stockout multiplier 3.0.

WHAT IT DOES NOT MEAN

- CHECK THIS: `RUNNING` is in the transition table but NO production code sets it on a session — only `jobs.status` goes RUNNING. The runner uses `force\_status`, which bypasses the machine entirely.
- CHECK THIS: the primary key of `session\_configs` is `session\_id` alone; `tenant\_id` is a column, not part of the key.
- CHECK THIS: `business.lead\_time\_days` is 7 in the runner and 15 in the wizard seed. The two defaults disagree.
- The data gate (`data\_gate.enforce`) blocks the LAUNCH, not the save, and has a single call site — which covers training, the demo, ERP sync and the seed script at once.

Service configuration

[backend/service_config/](#)

One registry from which the panel, `.env.example` and the reference are generated. Four precedence layers, and the panel says which one won for every field.

HOW IT WORKS

- Precedence: tenant override (channel services only) > instance override > environment > Settings default.
- A secret is Fernet-encrypted; with no key the write is REFUSED with a stated reason, never downgraded to plaintext.
- A read NEVER raises: if the database fails, the environment is reported and a .env-only deployment keeps working.
- Ten-second cache: a key pasted on the API container reaches the worker container without a restart.

WHAT IT DOES NOT MEAN

- Environment-only on purpose: the whole core (a UI that can rewrite its own database URL can lock everyone out) and the Fernet key (a panel that could rewrite it would make its own stored secrets unreadable).
- `WHATSAPP\_WEBHOOK\_BASE\_URL` is instance-only even though its service is tenant-scoped: the webhook verifies the signature BEFORE it knows which tenant the message belongs to.
- Cold start: while the deployment has EXACTLY ONE tenant, its admins operate it. It stops at two. An unreachable database grants nothing.

Public surface and the shape of errors

[backend/api/public_surface.py:26](#)

The published contract is eight (method, path) pairs. Everything else — 271 routes — is reachable with a key but nobody promised to keep it.

Success envelope

```
{"success": true, "data": ..., "meta": {"timestamp": ...}}
```

In 33 of 35 modules.

Error envelope

```
{"detail": "<English>", "error_code": "<stable snake_case>", "error_params": {...}}
```

The frontend renders ``errors.<error_code>`` interpolating the params; ``detail`` is the fallback.

PUBLIC_API_ONLY	Mounts the whole app and then PRUNES the routes to the public list plus /health. Pruning after mounting is deliberate: picking routers would let an internal neighbour ride along.
400 malformed_path	A NUL in the path is refused up front; it used to reach psycpg2 and surface as a 500.
503 server_busy	Connection pool exhausted, with Retry-After. Busy is not broken.
422 validation_error	With the echoed input sanitised: a NaN in it made Starlette turn the 422 into an empty-bodied 500.

WHAT IT DOES NOT MEAN

- PUBLIC_API_ONLY changes nothing about authentication or permissions: narrower reach, not a second security model. And it does NOT isolate the database — both instances share one Postgres.
- CHECK THIS: the route counts in the comments (246, 261) are stale; there are 271 decorators today.

The frontend

[Frontend/src/](#)

Next.js 14 App Router, 24 pages on Spanish routes, a 226-function API client, and three i18n catalogues with deliberately different shapes.

translations.ts	3,034 keys per language, runtime lookup. A typo returns the key.
landing.ts / serviceConfig.ts	Typed: a missing translation is a COMPILE error, which is the guarantee translations.ts needs a script to give.
Proxy	/api/:path* → \${BACKEND_URL}/api/v1/:path*. Note: the browser calls /api/... WITHOUT the v1; the proxy adds it.
BACKEND_URL	Defaults to 127.0.0.1:8010 — explicit IPv4, because Node resolves localhost to ::1 while uvicorn binds IPv4, producing a 500 with an empty body.

WHAT IT DOES NOT MEAN

- THERE ARE NO FRONTEND TESTS. None. `tsc --noEmit` checks types, not behaviour. What exists is three Playwright scripts run by hand.
- The Spanish routes are canonical; the old English ones 308-redirect permanently.
- Rewrites are evaluated at BUILD time: a Docker image must be built with `BACKEND_URL` set.